

JWT Authentication in the API Gateway

A class-by-class runtime trace for token creation, token validation, authorization, gateway routing, and signature verification.

PROJECT	SCOPE
Spring Boot 4.1 / Spring Cloud Gateway	Reactive WebFlux gateway on port 8080
JWT algorithm	HS256 using a shared secret
Authentication source	MapReactiveUserDetailsService + BCrypt
Authorization	ROLE_USER / ROLE_ADMIN endpoint rules

This document follows the code currently located in D:/OBPwork/api-gateway. Framework-managed steps are included because Spring Security registers them automatically when `oauth2ResourceServer().jwt()` is configured.

1. Complete class and function flow

This section intentionally contains only the ordered class and function names. Detailed explanations begin in Section 2.

A. Application startup

#	Class	Function / method
1	ApiGatewayApplication	main(String[] args)
2	SpringApplication	run(ApiGatewayApplication.class, args)
3	JwtProperties	<bind security.jwt>
4	SecurityConfig	jwtSecretKey(JwtProperties)
5	SecurityConfig	jwtEncoder(SecretKey)
6	SecurityConfig	jwtDecoder(SecretKey, JwtProperties)
7	SecurityConfig	jwtAuthenticationConverter()
8	SecurityConfig	passwordEncoder()
9	SecurityConfig	users(PasswordEncoder)
10	SecurityConfig	authenticationManager(ReactiveUserDetailsService, PasswordEncoder)
11	SecurityConfig	securityWebFilterChain(ServerHttpSecurity, ReactiveJwtAuthenticationConverter)

B. Login and JWT token creation

#	Class	Function / method
1	WebFilterChainProxy	filter(ServerWebExchange, WebFilterChain)
2	SecurityConfig	securityWebFilterChain(...) - permitAll /auth/token
3	AuthController	token(LoginRequest)
4	UsernamePasswordAuthenticationToken	unauthenticated(username, password)
5	UserDetailsRepositoryReactiveAuthenticationManager	authenticate(Authentication)
6	MapReactiveUserDetailsService	findByUsername(String)
7	BCryptPasswordEncoder	matches(rawPassword, encodedPassword)
8	JwtTokenService	issue(Authentication)
9	NimbusJwtEncoder	encode(JwtEncoderParameters)
10	AuthController.TokenResponse	TokenResponse(accessToken, tokenType, expiresIn)

C. JWT validation and authorization

#	Class	Function / method
1	WebFilterChainProxy	filter(ServerWebExchange, WebFilterChain)
2	ServerBearerTokenAuthenticationConverter	convert(ServerWebExchange)

#	Class	Function / method
3	JwtReactiveAuthenticationManager	authenticate(Authentication)
4	NimbusReactiveJwtDecoder	decode(String token)
5	ReactiveJwtAuthenticationConverter	convert(Jwt)
6	ReactiveJwtGrantedAuthoritiesConverterAdapter	convert(Jwt)
7	JwtGrantedAuthoritiesConverter	convert(Jwt)
8	AuthorizationWebFilter	filter(ServerWebExchange, WebFilterChain)
9	SecurityConfig	securityWebFilterChain(...) - authorizeExchange

D. After authorization: gateway and microservice

#	Class	Function / method
1	RequestLoggingFilter	filter(ServerWebExchange, GatewayFilterChain)
2	KeyResolver	resolve(ServerWebExchange)
3	RedisRateLimiter	isAllowed(String routeId, String id)
4	Spring Cloud Gateway	Path predicate + StripPrefix filter
5	ProductController	all() / byId(long)
6	OrderController	all() / byId(long) / create(CreateOrderRequest)

2. Step-by-step request navigation

Every HTTP request first reaches Reactor Netty on port 8080. Spring Security processes it before Spring Cloud Gateway chooses a downstream service. The request follows one of two paths: login/token creation or protected API access.

1. Gateway startup

`ApiGatewayApplication.main()` calls `SpringApplication.run()`. Spring scans the gateway packages, binds `JwtProperties`, builds the HS256 secret key, encoder, decoder, user store, authentication manager, role converter and `SecurityWebFilterChain`. Reactor Netty then listens on port 8080.

Result: The gateway is ready to receive HTTP requests.

2. Login request arrives

Postman sends `POST /auth/token`. The request first crosses Spring Security. The path is configured with `permitAll()`, so no existing JWT is required. Spring WebFlux then maps the request to `AuthController.token()` and converts the JSON body into `LoginRequest`.

Result: The request reaches the credential-authentication code.

3. Username and password authentication

`AuthController` creates an unauthenticated `UsernamePasswordAuthenticationToken` and passes it to `ReactiveAuthenticationManager.authenticate()`. The manager asks `MapReactiveUserDetailsService.findByUsername()` for the stored user, then calls `BCryptPasswordEncoder.matches()`. Roles come from trusted server-side user data, not from the login request.

Result: Correct credentials produce an authenticated object. Incorrect credentials produce 401.

4. JWT creation

`JwtTokenService.issue()` reads the authenticated username and authorities. It creates the issuer, subject, issued-at, expiration and roles claims. `NimbusJwtEncoder.encode()` signs the header and payload with HMAC-SHA256 using the gateway secret. `AuthController.TokenResponse` returns the signed access token to the client.

Result: 200 OK with `accessToken`, `tokenType=Bearer` and `expiresIn=1800`.

Protected request path

5. Bearer token extraction

For a protected call such as `GET /api/products/1`, Postman sends `Authorization: Bearer <JWT>`. A Spring Security bearer-token converter extracts the token. This framework component was registered automatically by `oauth2ResourceServer().jwt()`.

Result: A missing or malformed Bearer token produces 401 before routing.

6. JWT signature and claim validation

`JwtReactiveAuthenticationManager` calls `NimbusReactiveJwtDecoder.decode()`. The decoder recalculates HMAC-SHA256 over the received header and payload using the gateway's secret. It compares that calculated value with the signature carried in the third JWT segment. It also checks expiration, time validity and issuer.

Result: A valid token continues. A changed, expired, malformed or wrong-issuer token produces 401.

7. JWT becomes an authenticated principal

`ReactiveJwtAuthenticationConverter.convert()` creates the Spring Security authentication object. `JwtGrantedAuthoritiesConverter.convert()` reads the `roles` claim and adds the `ROLE_` prefix. Therefore `USER` becomes `ROLE_USER` and `ADMIN` becomes `ROLE_ADMIN`. The JWT subject becomes the principal name used by logging and rate limiting.

Result: Spring now considers the request authenticated.

8. Authorization decision

The authorization rules in `SecurityConfig.securityWebFilterChain()` evaluate the HTTP method and path. Any valid user can access product and GET order endpoints. `POST /api/orders/**` requires `ROLE_ADMIN`. Authentication asks who the caller is; authorization asks whether that identity has permission for this endpoint.

Result: Insufficient role produces 403. A permitted request enters the gateway route chain.

9. Gateway filters and routing

`RequestLoggingFilter.filter()` adds or preserves `X-Correlation-Id`. The rate limiter resolves the authenticated username and checks Redis. Spring Cloud Gateway then matches the path predicate. `/api/products/**` selects Product Service; `/api/orders/**` selects Order Service. `StripPrefix` removes the first `/api` segment before forwarding.

Result: The downstream controller receives an internal HTTP request.

10. Downstream response returns

Spring invokes `ProductController.all()/byId()` or `OrderController.all()/byId()/create()`. The microservice response travels back through the gateway. A product response may be stored in the local Caffeine cache. The logging filter records route, user, status and duration, and Netty returns the final response to Postman.

Result: The client receives the microservice result through port 8080.

3. Authentication and authorization cases

1. Valid login

The submitted username exists and BCrypt matches the password hash. The authenticated object contains server-side roles, so `JwtTokenService` signs and returns a JWT.

Result: 200 OK + JWT

2. Invalid login

The username is not found or BCrypt does not match the password. `AuthController` maps `AuthenticationException` to an unauthorized response. `JwtTokenService` is never called.

Result: 401 Unauthorized

3. Missing token

A protected request has no Bearer token. Spring Security cannot create an authenticated principal, so the request stops before gateway routing.

Result: 401 Unauthorized

4. Valid USER token

The decoder validates the JWT and the roles claim becomes `ROLE_USER`. Product endpoints and GET order endpoints require only an authenticated identity.

Result: Request allowed

5. USER posts an order

The JWT is valid, so authentication succeeds. Authorization then checks `hasRole(ADMIN)`, but the principal has only `ROLE_USER`.

Result: 403 Forbidden

6. ADMIN posts an order

The JWT validates and contains `ADMIN`. The converter produces `ROLE_ADMIN`, the authorization rule passes, and the request is routed to `OrderController.create()`.

Result: 201 Created

7. Expired or wrong-issuer token

`NimbusReactiveJwtDecoder` verifies time claims and issuer after cryptographic validation. An expired token or an issuer different from `demo-api-gateway` is rejected.

Result: 401 Unauthorized

8. Tampered payload

Changing `USER` to `ADMIN` changes the encoded payload. The token still carries the old signature. The gateway recalculates a new signature from the changed data and its local secret; the two signatures do not match.

Result: 401 Unauthorized

9. Rate limit exceeded

Authentication and authorization succeed first. `RedisRateLimiter` then finds that the user's token bucket has insufficient tokens, so the call does not reach the microservice.

Result: 429 Too Many Requests

How signature comparison works

```
originalData = base64Url(header) + '.' + base64Url(originalPayload)
receivedSignature = HMAC-SHA256(secret, originalData)

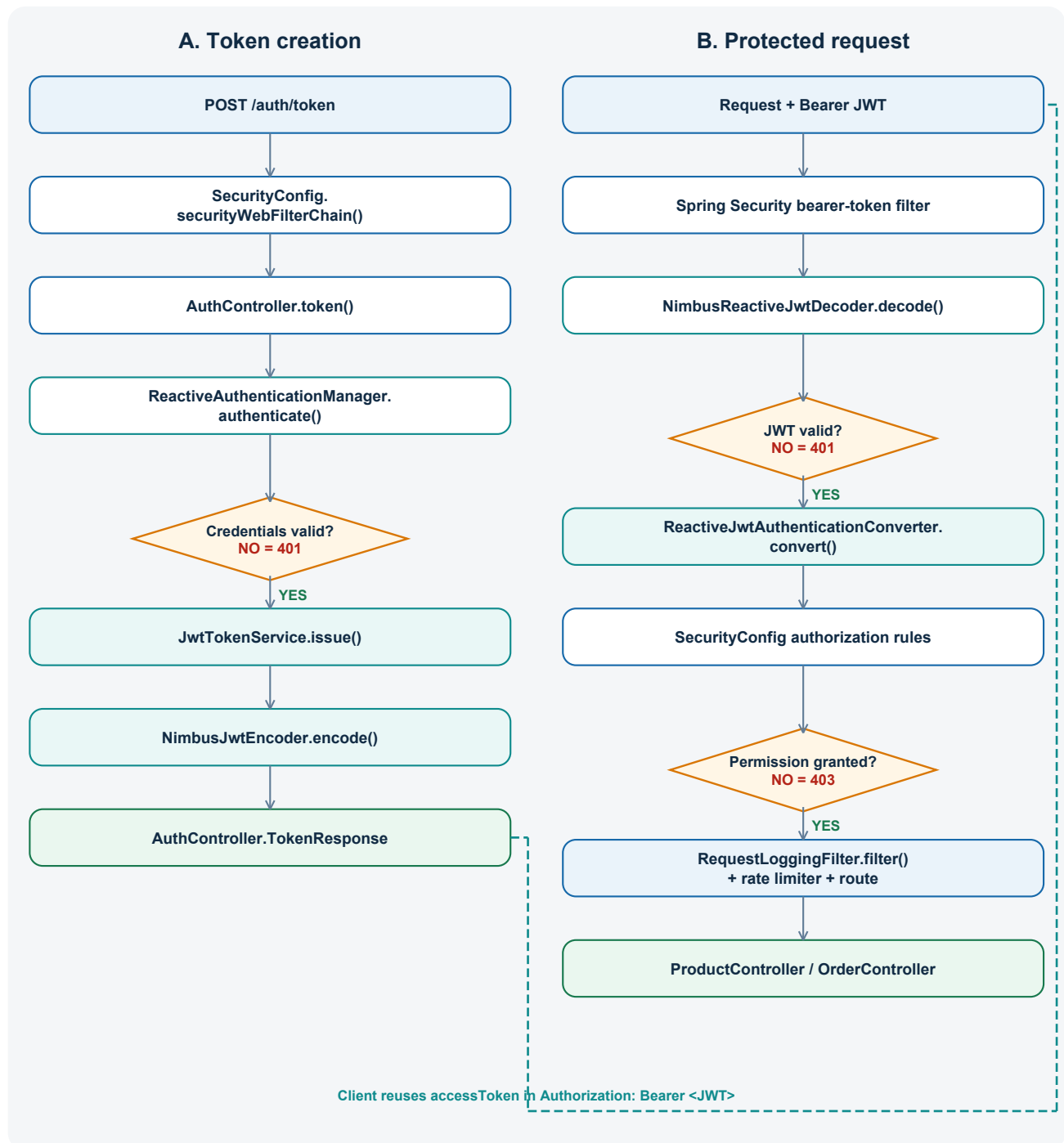
changedData = base64Url(header) + '.' + base64Url(changedPayload)
calculatedSignature = HMAC-SHA256(secret, changedData)

changedData != originalData => calculatedSignature != receivedSignature
```

The signature is the third part sent by the client. The calculated signature is generated locally by the gateway from the first two parts and its secret. The secret is never placed inside the JWT. Base64Url encoding is readable encoding, not encryption.

4. End-to-end JWT request flow

The diagram combines token creation and protected-request processing. Each decision diamond shows the rejection status, while the YES path continues toward token issuance or the downstream microservice.



5. Key implementation notes

1. No custom JWT filter

The project does not define a `JwtAuthenticationFilter`. Spring Security creates the reactive bearer-token filter chain because `oauth2ResourceServer().jwt()` is configured.

2. Passwords are used only at login

A protected request is authenticated by validating the JWT. The gateway does not check the username and password again for every API request.

3. JWT roles are trusted only after validation

The roles claim is not accepted until the signature, issuer and timestamps have been validated by `NimbusReactiveJwtDecoder`.

4. Authentication comes before routing

A rejected request never reaches `RequestLoggingFilter`, Redis rate limiting, the route predicate or a downstream controller.

5. Current demo is stateless

Once issued, the token remains usable until expiration unless the signing secret changes. The in-memory user store is not consulted during normal JWT validation.

6. Gateway is the security boundary

Product Service and Order Service do not independently validate the JWT. They must not be publicly reachable in a production network.

Source files followed

- `api-gateway/src/main/java/com/example/gateway/ApiGatewayApplication.java`
- `api-gateway/src/main/java/com/example/gateway/security/SecurityConfig.java`
- `api-gateway/src/main/java/com/example/gateway/security/AuthController.java`
- `api-gateway/src/main/java/com/example/gateway/security/JwtTokenService.java`
- `api-gateway/src/main/java/com/example/gateway/security/JwtProperties.java`
- `api-gateway/src/main/java/com/example/gateway/config/RateLimitConfig.java`
- `api-gateway/src/main/java/com/example/gateway/filter/RequestLoggingFilter.java`
- `api-gateway/src/main/resources/application.yml`
- `product-service/src/main/java/com/example/product/ProductController.java`
- `order-service/src/main/java/com/example/order/OrderController.java`